

Towards Retargetable Animations for Industrial Augmented Reality

Reza Mirzaiee
University of Virginia

Teodor Vernica
Aarhus University

Kurt Scheuringer
Lockheed Martin Corporation

William Z. Bernstein*
Air Force Research Laboratory

ABSTRACT

The adoption of Augmented Reality (AR) within manufacturing has surged. However, updating animations due to upstream changes or design variations is expensive. Currently, platform-agnostic standards for computer-aided design files do not adequately specify kinematic animations. Consequently, animations require manual, individual updates. We showcase a method for implementing geometry-independent AR assembly animations using skeletal armatures, a technique widely used in the entertainment industry. This technique allows upstream engineering changes to propagate through to the AR assembly visualization, leading to a more automated pipeline for handling animations in Industrial AR systems.

Index Terms: Human-centered computing—Human computer interaction (HCI)—Interaction paradigms—Mixed / augmented reality

Keywords: Industrial animations; Industry 4.0; Manufacturing

1 MOTIVATION AND BACKGROUND

Augmented reality (AR) has experienced a recent surge in adoption and integration within the manufacturing industrial base. By registering animated assembly instructions in three dimensions, information can be contextualized and brought closer to where it is needed, compared to traditional 2D manuals. In this sense, studies show AR guidance can reduce cognitive load, decrease the number of errors and shorten task completion times. For example, Hou et al. [1] found that an animated AR system can shorten task completion times, and learning curves can be lessened for training novice assemblers. Previous example use cases for AR-assisted assembly include car door-lock assembly [2], training of novice users for computer motherboard assembly [3], installation of a Computer Numerical Control (CNC) carving machine [4], and others [5].

Central to applications designed for AR-assisted assembly are animations. Animations can serve as dynamic instructions for operators. Coupled with computer vision, instructions can be delivered automatically in the same spatial context of the physical assembly [6].

Encoding animations for procedural task guidance is an essential component of the development workflow. Yet, its workflow remains fragmented and complex. Figure 1 provides an overview of the traditional AR-assisted assembly software pipeline. First, computer-aided design (CAD) models, including their final state of assembly constraints, and supporting documentation, e.g., bills of materials (BOMs), are developed by engineering teams. Afterwards, an internal AR team or external consultant is responsible for collecting all relevant information and reconstructing the models into an AR-capable representation, such as glTF, within an AR game engine. This team often comprises of a wide range of skills and professions, including software developers, artists, and industrial designers. The original CAD model geometry is usually decimated and optimized into a lightweight representation [7]. Next, the AR team needs to interpret the assembly instructions and create sequential animations

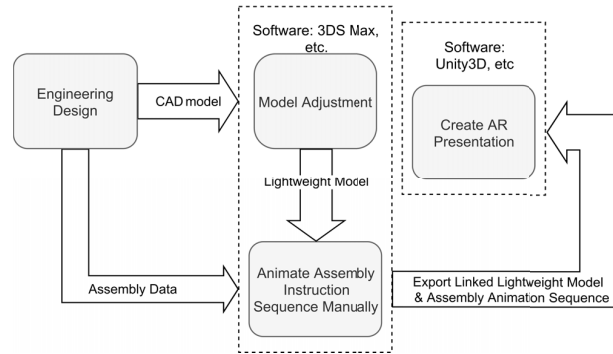


Figure 1: Traditional AR-assisted assembly software pipeline.

for them, since platform-agnostic standards do not specify kinematic animations adequately. These animations are then encoded into lightweight representations, ultimately binding together the AR geometry and assembly animations. Finally, the AR team exports the file into an AR game engine, such as Unity3D, and creates an application that can be deployed onto an AR-capable device, such as a tablet or head-mounted display.

AR animation development is unidirectional and does not directly afford channels for upstream changes without breaking subsequent phases. In practice, engineering change is a common occurrence. When a change is ordered, the authoritative 3D CAD model might change, which would require a new lightweight representation. Since the animation data is directly embedded within the lightweight model, animations must also be re-tooled. In other words, after each change request, the authoritative models must be processed through the AR pipeline. To overcome this bottleneck, practitioners have developed in-house solutions to better react to such disruptions. However, no platform-agnostic, standard solution currently exists.

In response, we propose a conceptual solution to facilitate more update-able, maintainable, and flexible industrial animations for AR-assisted assembly. Our approach adopts the popular concept of skeletal animation. A “skeletal animation rig” is a hierarchy of 3D points that deforms a 3D models to which it is bound to, forming a sort of mannequin. By translating these 3D points in space, an animation artist is able to deform a 3D model and “pose” it; poses can be recorded in sequence over a period of time to create the illusion of movement. A skeletal animation rig is a separate entity from the model to which it is bound; one is able to re-purpose the same rig (and thus, the same animation) and bind to many different models as long as they share similar features. In many cases, a skeletal rig animation may be deployed to any humanoid [8,9].

Expediting 3D animation development, skeletal animation re-targeting has been a revolutionary concept in gaming industries. Widely adopted humanoid skeleton definitions and standards have driven the development of resources, such as libraries of high quality geometry-independent humanoid animations [10].

Our contributions can be summarized as (i) an articulation of a framework for retargetable industrial animations, (ii) case study of showing the repurposing of an animation across a family of screws, and (iii) an AR demonstration showing the utility of the framework.

* Address all correspondence to this author
e-mail: william.bernstein@us.af.mil

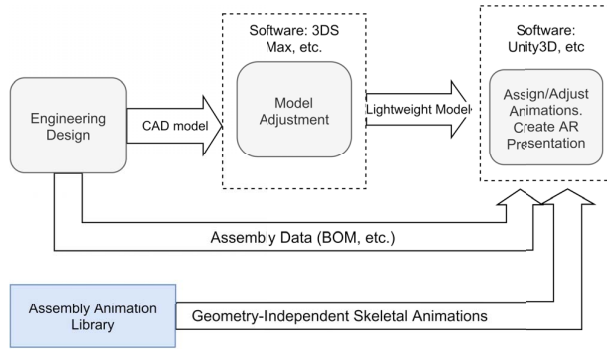


Figure 2: Proposed AR-assisted assembly software pipeline.

2 TECHNICAL APPROACH

In manufacturing, many assembly components can be described as “hard surface models,” made of stiff materials. To support this context, skeletal rigging conventions require changes, as the goal is to translate and rotate models through space rather than deform them along joints. To this end, we define animations as transformations in local space, or its “Root Movement.” Since we do not need to deform models, we only need to define a single bone in our skeleton – the root of the skeletal rig. From here, we can create animations by moving the skeletal root from its origin to wherever the part must move. For example, screws might move downwards along vertical axes and rotate clockwise. In such cases, we animate a placeholder along a motion path. Later, we import these animations into a game engine and apply our generic animation to any model. Through this procedure, we decouple the animation data from the 3D models and directly import them into the final AR presentation. Thus, retooling animations after changes to the authoritative models can be bypassed, since the animation is defined within the game engine agnostic of the solid models. Figure 2 illustrates our proposed AR-assisted assembly software pipeline, including the possibility of retargetable animation libraries for industrial AR assemblies.

Challenges exist to this method. First, a standard for defining the 3D model’s coordinate origin needs to be established. Typically, when practitioners create assembly animations, they define each component’s movement relative to a coordinate space common to all other models. This implies that animations are context-dependent; a screw angled at forty-five degrees will move differently than a screw angled at ninety degrees relative to each other. Hence, bone animations that use a global coordinate space are not retargetable, since the motion will depend on the object’s orientation and placement (rotation and position). However, the “root movement” is equivalent as long as we use an origin point in the object’s local coordinate space. For example, from a screw’s reference point, a screw will always move downwards and tighten to the right. Through the use of locally-defined animations, we may generalize the motion and adjust it later in the pipeline to fit project-specific needs.

In our case study, we retarget a single skeletal animation to a family of screws, defining our skeletal animation’s origin as center-point of the 2D plane in which the screw makes contact with a surface when it is fully secured. In the future, local coordinate origins need to be standardized for each class of assembly components to ensure a library of skeletal animations is correctly applied.

Another challenge is the task of recontextualizing reusable animations onto a specific assembly. Because the skeletal animations are defined locally, developers must place models in the correct starting locations and orientations. In our AR-assisted assembly demo (Figure 3), this was done manually. This process may prove

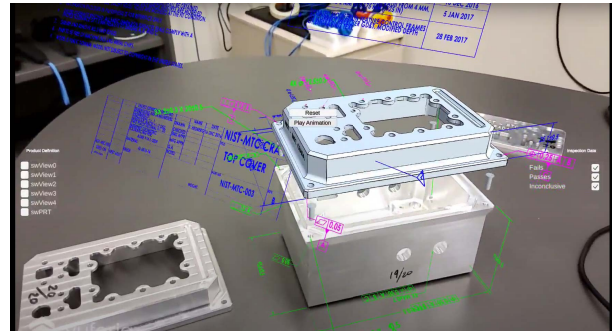


Figure 3: AR-assisted assembly demo, using PMI annotations [7].

to be labor-intensive, given that physical changes to an assembly might require manual adjustment of the imported skeletal animations to fit the modified components. Additionally, there may be cases where generic animations are not specific enough for a particular project. In these cases, the original skeletal animations may need to be modified, or a specialized, specific animation needs to be defined.

3 FUTURE WORK

We intend to better define and evaluate local coordinate origin standards for a larger variety of assembly components. This will lead to the ability to showcase animation reusability in and across more complex industrial assembly tasks. Secondly, we plan to extend our framework to support automated, procedural placement of animations, avoiding the need for adjustments in the AR engine stage of the pipeline. For example, developers could import each model’s correct placement from a file parsed into the AR engine. This file would match components in the assembly to their corresponding coordinate origin standards, and automatically line-up animations based on 3D spatial information from the CAD file. Lastly, we intend to improve the flexibility of our framework by parameterizing industrial animations, allowing for configurable motions that could be adjusted to meet specific needs, with the possibility of automating the adjustment process as well. A previous study [11] demonstrated robust animation retargeting for highly variable models.

REFERENCES

- [1] L. Hou *et al.*, “Using animated augmented reality to cognitively guide assembly,” *J Comput Civ Eng*, vol. 27, no. 5, pp. 439–451, 2013.
- [2] D. Reiners *et al.*, “Augmented reality for construction tasks: Door-lock assembly,” *Augmented Reality: Placing Artificial Objects in Real Scenes*, vol. 98, no. 1, pp. 31–46, 1999.
- [3] G. Westerfield *et al.*, “Intelligent augmented reality training for motherboard assembly,” *Int J Artif Intell Educ*, vol. 25, no. 1, pp. 157–172, 2015.
- [4] Z.-H. Lai *et al.*, “Smart augmented reality instructional system for mechanical assembly towards worker-centered intelligent manufacturing,” *Journal of Manufacturing Systems*, vol. 55, pp. 69–81, 2020.
- [5] S. K. Ong *et al.*, “Augmented reality applications in manufacturing: a survey,” *Int J Prod Res*, vol. 46, no. 10, pp. 2707–2742, 2008.
- [6] T. Salonen *et al.*, “Data pipeline from CAD to AR based assembly instructions,” in *ASME world conference on innovative virtual reality*, vol. 43376, 2009, pp. 165–168.
- [7] T. Vernica *et al.*, “Visualizing model-based product definitions in augmented reality,” *Proc. of the ASME IDETC/CIE 2021*, 2021.
- [8] A. Feng *et al.*, “Fast, automatic character animation pipelines,” *Computer Animation and Virtual Worlds*, 2013.
- [9] M. Borno *et al.*, “Robust physics-based motion retargeting with realistic body shapes,” *Computer Graphics Forum*, vol. 37, pp. 81–92, 12 2018.
- [10] “Mixamo,” <https://www.mixamo.com>, accessed: 2021-06-10.
- [11] C. Hecker *et al.*, “Real-time motion retargeting to highly varied user-created morphologies,” *ACM*, 2008.